ProxiSpeak — Server Code

Node.js · Express · Socket.IO · MongoDB · Optional Redis

Save as: `server/index.js`

Note: This PDF contains the code you provided, formatted for reading and printing.

```
import express from 'express';
import http from 'http';
import cors from 'cors';
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';
import { createClient } from 'redis';
import { createAdapter } from '@socket.io/redis-adapter';
import dotenv from 'dotenv';
import { Server } from 'socket.io';
import { MongoClient } from 'mongodb';

dotenv.config();

const PORT = Number(process.env.PORT || 4000);
const CLIENT_ORIGIN =
  process.env.CLIENT_ORIGIN || 'http://localhost:5173';

const app = express();

app.disable('x-powered-by');
app.use(helmet({ crossOriginEmbedderPolicy: false }));
app.use(cors({ origin: CLIENT_ORIGIN }));

app.use(
  '/api/',
  rateLimit({
    windowMs: 60_000,
    limit: 120,
    standardHeaders: 'draft-7',
    legacyHeaders: false,
  })
);

app.use(express.json({ limit: '1mb' }));

const httpServer = http.createServer(app);

const io = new Server(httpServer, {
  cors: {
    origin: CLIENT_ORIGIN,
    methods: ['GET', 'POST'],
  },
});

// MONGODB

let mongoClient;
let positionsCollection;
let roomsCollection;

if (process.env.MONGODB_URI) {
  try {
    mongoClient = new MongoClient(process.env.MONGODB_URI);
    await mongoClient.connect();

    const db = mongoClient.db('proxispeak');

    positionsCollection = db.collection('positions');
    roomsCollection = db.collection('rooms');

    await positionsCollection.createIndex(
      { roomId: 1, socketId: 1 },
      { unique: true }
    );

    await roomsCollection.createIndex(
      { roomId: 1 },
      { unique: true }
    );

    console.log('MongoDB connected; indexes ready');
  } catch (error) {
    console.error(
      'MongoDB setup failed; continuing in-memory:',
      error.message
    );

    positionsCollection = null;
    roomsCollection = null;
  }
}
```

```
// USERS AND VALIDATION

const users = new Map();
let redisClients = null;

const roomUsers = (roomId) =>
  [...users.values()].filter((user) => user.roomId === roomId);

const validCoord = (value, max) => {
  if (typeof value !== 'number' || !Number.isFinite(value)) {
    return null;
  }

  return Math.max(24, Math.min(max, value));
};

// REDIS ADAPTER (OPTIONAL)

if (process.env.REDIS_URL) {
  try {
    const pubClient = createClient({
      url: process.env.REDIS_URL,
    });

    const subClient = pubClient.duplicate();

    pubClient.on('error', (error) =>
      console.error('Redis publisher error:', error.message)
    );

    subClient.on('error', (error) =>
      console.error('Redis subscriber error:', error.message)
    );

    await Promise.all([
      pubClient.connect(),
      subClient.connect(),
    ]);

    io.adapter(createAdapter(pubClient, subClient));

    redisClients = [pubClient, subClient];

    console.log('Socket.IO Redis adapter enabled');
  } catch (error) {
    console.error(
      'Redis adapter unavailable; using single-instance Socket.IO:',
      error.message
    );
  }
}

// HTTP API ROUTES

app.get('/api/health', (_req, res) => {
  res.json({
    ok: true,
    service: 'ProxiSpeak',
    connectedUsers: users.size,
    persistence: Boolean(positionsCollection),
    redisAdapter: Boolean(redisClients),
  });
});

app.get('/api/ready', (_req, res) => {
  if (process.env.MONGODB_URI && !positionsCollection) {
    return res.status(503).json({
      ready: false,
      reason: 'MongoDB unavailable',
    });
  }

  res.json({ ready: true });
});

app.get('/api/users', (req, res) => {
  const roomId = String(
    req.query.roomId || 'main-office'
  ).slice(0, 40);

  res.json(roomUsers(roomId));
});

app.get('/api/rooms/:roomId', async (req, res) => {
  try {
    const roomId = String(req.params.roomId).slice(0, 40);

    const saved = roomsCollection
      ? await roomsCollection.findOne({ roomId })
      : null;

    res.json(
      saved || {
```

```
        roomId,
        layout: 'default',
        persisted: false,
      }
    );
  } catch (error) {
    res.status(500).json({
      error: 'Could not load room layout',
    });
  }
});

app.put('/api/rooms/:roomId', async (req, res) => {
  if (!roomsCollection) {
    return res.status(503).json({
      error: 'Configure MONGODB_URI to persist room layouts',
    });
  }

  const roomId = String(req.params.roomId).slice(0, 40);
  const layout = req.body?.layout;

  if (!layout || typeof layout !== 'object' || Array.isArray(layout)) {
    return res.status(400).json({
      error: 'layout object required',
    });
  }

  try {
    await roomsCollection.updateOne(
      { roomId },
      {
        $set: {
          roomId,
          layout,
          updatedAt: new Date(),
        },
      },
      { upsert: true }
    );

    res.json({ ok: true, roomId });
  } catch (error) {
    res.status(500).json({
      error: 'Could not save room layout',
    });
  }
});

// SOCKET.IO EVENTS

io.on('connection', (socket) => {
  socket.on('office:join', async (payload = {}) => {
    if (users.has(socket.id)) {
      socket.emit('office:join:error', {
        message: 'You have already joined a room.',
      });
      return;
    }

    if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
      socket.emit('office:join:error', {
        message: 'Invalid join request.',
      });
      return;
    }

    const { name, roomId } = payload;

    const safeName =
      String(name || 'Guest').trim().slice(0, 24) || 'Guest';

    const safeRoom =
      String(roomId || 'main-office').trim().slice(0, 40) ||
      'main-office';

    const user = {
      id: socket.id,
      name: safeName,
      roomId: safeRoom,
      x: 120 + Math.random() * 480,
      y: 100 + Math.random() * 280,
      joinedAt: new Date().toISOString(),
    };

    socket.data.roomId = safeRoom;
    socket.join(safeRoom);

    users.set(socket.id, user);

    if (positionsCollection) {
      positionsCollection
        .updateOne(
```

```
        { roomId: safeRoom, socketId: socket.id },
        {
          $set: {
            ...user,
            socketId: socket.id,
            updatedAt: new Date(),
          },
        },
        { upsert: true }
      )
      .catch((error) =>
        console.error('Mongo position save failed:', error.message)
      );
  }

  socket.emit('office:joined', {
    selfId: socket.id,
    users: roomUsers(safeRoom),
  });

  socket.to(safeRoom).emit('user:joined', user);

  io.to(safeRoom).emit('users:update', roomUsers(safeRoom));
});

socket.on('avatar:move', (payload) => {
  if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
    return;
  }

  const user = users.get(socket.id);

  if (!user) return;

  const nextX = validCoord(payload.x, 776);
  const nextY = validCoord(payload.y, 476);

  if (nextX === null || nextY === null) {
    socket.emit('avatar:move:error', {
      message: 'Invalid movement coordinates.',
    });
    return;
  }

  user.x = nextX;
  user.y = nextY;

  users.set(socket.id, user);

  if (positionsCollection) {
    positionsCollection
      .updateOne(
        { roomId: user.roomId, socketId: socket.id },
        {
          $set: {
            x: user.x,
            y: user.y,
            updatedAt: new Date(),
          },
        }
      )
      .catch((error) =>
        console.error('Mongo position update failed:', error.message)
      );
  }

  io.to(user.roomId).emit('avatar:moved', user);
});

// AUDIO SIGNALING

socket.on('audio:signal', (payload = {}) => {
  if (!payload || typeof payload !== 'object') return;

  const { targetId, signal } = payload;

  if (!targetId || !signal) return;

  const sender = users.get(socket.id);
  const target = users.get(targetId);

  if (!sender || !target) return;

  if (sender.roomId !== target.roomId) return;

  io.to(targetId).emit('audio:signal', {
    fromId: socket.id,
    signal,
  });
});

// DISCONNECT AND CLEANUP
socket.on('disconnect', () => {
```

```
    const user = users.get(socket.id);

    if (!user) return;

    users.delete(socket.id);

    if (positionsCollection) {
      positionsCollection
        .deleteOne({
          roomId: user.roomId,
          socketId: socket.id,
        })
        .catch((error) =>
          console.error('Mongo position cleanup failed:', error.message)
        );
    }

    io.to(user.roomId).emit('user:left', {
      id: socket.id,
    });

    io.to(user.roomId).emit('users:update', roomUsers(user.roomId));
  });
});

// START SERVER

httpServer.listen(PORT, () => {
  console.log(
    `ProxiSpeak server running at http://localhost:${PORT}`
  );
});
```